Universidad de Costa Rica Sede Regional del Pacífico Bach.
Informática Empresarial
Desarrollo de Software IV
Proyecto integrador
Fecha de entrega: 29/10/2026

Desarrollo de una API REST con Django

Documento de asignación académica. Cada equipo propone su propio problema y diseña su solución; no se proporciona un proyecto resuelto.

1. Propósito y alcance

Diseñar, implementar, probar y documentar un sistema backend que atienda una necesidad real o verificable de una organización, comunidad o sector productivo. El proyecto deberá desarrollarse con base en los contenidos, conceptos, prácticas y herramientas abordados en clase. La solución deberá integrar al menos cinco módulos funcionales de negocio relacionados, además de autenticación y autorización. No se requiere desarrollar frontend: la evaluación funcional se realizará mediante Postman y pruebas automatizadas.

2. Organización y tecnologías

Aspecto	Condición
Equipo	Tres integrantes; responsabilidades identificables y revisión cruzada del código.
Backend	Python, Django y Django REST Framework.
Persistencia	MySQL o PostgreSQL; Django ORM y migraciones versionadas.
Seguridad	Autenticación por tokens, roles y permisos por operación.
Pruebas y documentación	Postman, pruebas automatizadas, README y documentación de API.
Colaboración	Git y GitHub con ramas, pull requests o revisiones y commits atribuibles.

3. Elección y aprobación del problema

Cada equipo seleccionará un contexto de aplicación, identificará usuarios y describirá una necesidad concreta. No se aceptarán cinco CRUD independientes sin un proceso de negocio común. La propuesta deberá mostrar cómo circulan los datos entre módulos y qué decisiones automatiza el sistema. El alcance se ajustará a la etapa disponible y a las capacidades del equipo.

Ejemplos orientativos de dominios: gestión de servicios, inventario y abastecimiento, turismo comunitario, eventos, emprendimientos, atención de solicitudes o gestión académica. Son referencias, no temas obligatorios.

4. Requisitos mínimos verificables

ID	Requisito	Evidencia / aceptación
RF-01	Cinco módulos de negocio	Al menos cinco aplicaciones o componentes de negocio con

		responsabilidades delimitadas; describir entradas, salidas y relaciones.
RF-02	CRUD y persistencia	CRUD completo en al menos cuatro módulos; justificar cualquier operación omitida en el quinto.
RF-03	Modelo relacional	Claves primarias, foráneas, al menos una relación 1:N y una N:M cuando sea pertinente; restricciones únicas y políticas de borrado justificadas.
RF-04	Consultas ORM	Al menos cinco consultas no triviales; incluir agregación, filtros por relaciones y una consulta con select_related/prefetch_related cuando aplique.
RF-05	Consulta REST	Filtros combinables en dos recursos, búsqueda en uno, ordenamiento en uno y paginación en al menos dos listados.
RF-06	Validación	Al menos ocho reglas de negocio documentadas, con respuestas 400 y pruebas negativas; proteger invariantes también en PATCH.
RF-07	Seguridad	Al menos tres perfiles (p. ej. administrador, operador, consulta); matriz de permisos; 401 y 403 comprobados.
RF-08	Errores HTTP	Usar 200/201/204/400/401/403/404/409

		según corresponda; respuestas coherentes sin trazas internas.
RF-09	Calidad y entrega	Documentación de endpoints, colección Postman reproducible, migraciones, pruebas automatizadas y repositorio ejecutable.

5. Desarrollo, pruebas y entrega final

Fase	Actividades y evidencia requerida
1. Preparación	Repositorio, entorno virtual, dependencias, configuración por .env, base de datos y estructura de aplicaciones Django.
2. Persistencia	Modelos, restricciones, migraciones, datos ficticios y comprobación de relaciones en la base de datos.
3. API de negocio	Serializadores, vistas, rutas, CRUD, filtros, búsquedas, ordenamiento, paginación y respuestas HTTP.
4. Seguridad	Usuarios de prueba, tokens, roles, permisos por endpoint y casos de acceso denegado.
5. Pruebas	Al menos 13 pruebas automatizadas: 5 casos CRUD, 4 reglas de negocio, 3 permisos y 1 caso de paginación o filtro.
6. Documentación	README reproducible, .env.example, requirements.txt, migraciones, colección Postman, catálogo final de endpoints y guía de ejecución.
7. Demostración	Ejecución desde instalación limpia o entorno preparado, carga de datos, demostración de

	cinco módulos, seguridad y respuestas a preguntas individuales.

6. Evidencias técnicas obligatorias

1. Repositorio GitHub accesible al docente, con historial de ramas, commits, pull requests/revisiones y contribuciones de cada integrante.
2. Código fuente y migraciones; no incluir .env, tokens, contraseñas, datos reales ni carpetas del entorno virtual.
3. README con requisitos, instalación, creación de base de datos, migraciones, superusuario, ejecución, pruebas y limitaciones conocidas.
4. Archivo .env.example sin secretos; requirements.txt o archivo equivalente con versiones reproducibles.
5. Colección Postman exportada, variables de entorno de ejemplo y al menos 20 solicitudes documentadas con respuestas esperadas.
6. Evidencia de ejecución de al menos 15 pruebas automatizadas y explicación de los casos no superados, si existieran.
7. Documento técnico final con diagrama ER actualizado, matriz de permisos, reglas de negocio y catálogo de endpoints.

7. Rúbrica de evaluación — 100 puntos

La evaluación combina 70 puntos grupales y 30 individuales.º

Modalidad	Criterio	Puntos
Grupal	Modelos, migraciones e integridad relacional	8
Grupal	Endpoints CRUD, consultas, filtros y paginación	10

Grupal	Validaciones, autenticación, roles y errores HTTP	9
Grupal	Pruebas automatizadas, Postman y documentación	6
Individual	Contribuciones verificables y revisión de código	10
Individual	Defensa técnica y comprensión de los cinco módulos	12
Individual	Demostración individual de implementación, pruebas y resolución de incidencias	8

Comprobación: cada integrante presentará al menos dos contribuciones sustantivas revisables y defenderá una parte implementada por otra persona. Los commits por sí solos no garantizan autoría: el docente contrastará código, revisiones, tablero y defensa oral.

8. Demostración final

Cada equipo dispondrá de una demostración guiada y preguntas individuales. El docente podrá seleccionar endpoints y reglas de negocio al azar, solicitar una operación con usuario no autorizado, modificar un dato para comprobar una validación y ejecutar una prueba automatizada. Los tres estudiantes deben poder explicar el flujo solicitud → vista → serializador/servicio → ORM → respuesta.

9. Criterios de entrega y consideraciones

La entrega se considerará completa cuando el docente pueda clonar el repositorio, instalar dependencias, configurar variables de entorno, ejecutar migraciones, iniciar el servidor, importar la colección Postman y reproducir los casos principales.

Se evaluará la solución backend y su justificación técnica, no la complejidad visual de una interfaz. El plagio de proyectos ajenos, la publicación de credenciales o la fabricación de evidencias se gestionarán conforme a la normativa académica aplicable. Deben subir un archivo txt con el enlace de github en mediacionvirtual.ucr.ac.cr